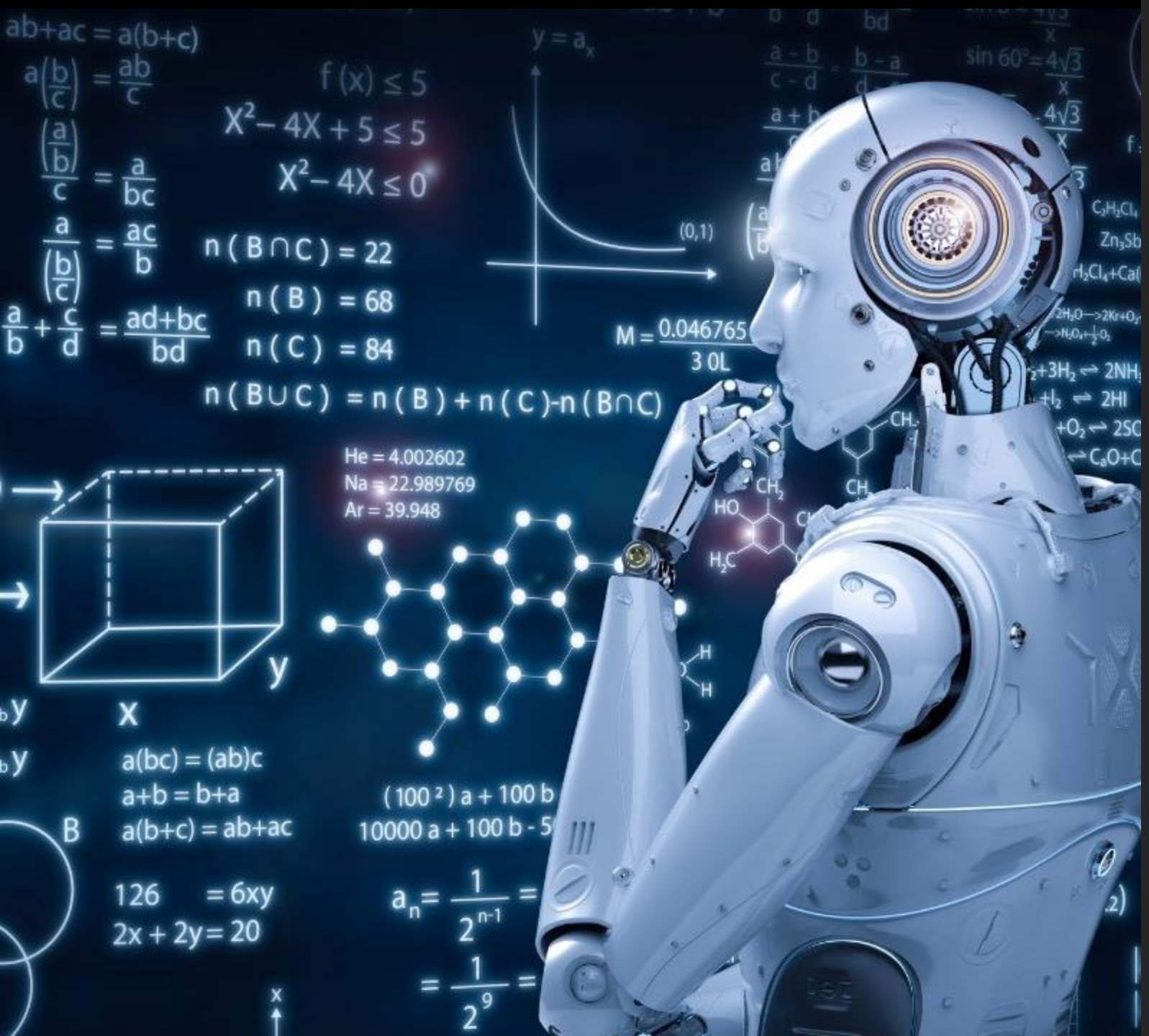



# Learning Outcome 5

**UNDERSTANDING ALGORITHM EFFICIENCY**

PROF. DR. RASHMI GUPTA

RASHMI.GUPTA@KRISTIANIA.NO

# Major Objectives

- Complexity Analysis of Algorithms
- What is Time Complexity?
- What is Space Complexity?
- What are Asymptotic Notations?
- How to calculate Time/Space Complexity?
- Types of Time Complexity?

# Complexity Analysis of Algorithms

- Here we explore **computational complexity (space and time complexity)**, developed by Juris Hartmanis and Richard E. Stearns to assess the difficulty of an algorithm.
- As we discussed, human nature strives to find the most efficient way to complete their daily tasks. The overarching thought process behind innovation and technology is to make people's lives easier by providing solutions to problems they may face.
- In the world of computer science and digital products, the same thing occurs. To perform better, we need to write algorithms that are **time-efficient and use less memory**.

# What is Time complexity??

- Time complexity is defined in terms of **how many times it takes to run a given algorithm, based on the length of the input.**
- Time complexity is **not a measurement of how much time it takes to execute a particular algorithm** because such factors as programming language, operating system, and processing power are also considered.
- Time complexity is a type of **computational complexity** that describes the time required to execute an algorithm.
- The time complexity of an algorithm is the amount of time it takes for each statement to complete.
- As a result, it is **highly dependent on the size of the processed data.** It also aids in defining an algorithm's effectiveness and evaluating its performance.

# What is Space Complexity??

- When an algorithm runs on a computer, it necessitates a certain amount of memory space.
- The **amount of memory used by a program to execute** it is represented by its space complexity.
- Because a program requires **memory to store input data and temporal values while running,** the space complexity is **auxiliary and input space.**

**Auxiliary memory** refers to the **extra memory** an algorithm needs **in addition to the input data** during execution. It is temporary memory used for **intermediate calculations, recursion, or storing temporary results.**

- **Auxiliary memory is part of space complexity,** but it does **not** include the memory needed to store the input itself.

# Method to Calculate Time Complexity

```
1.  mul <- 1
2.  i <- 1
3.  While i <= n do
4.  mul = mul * 1
5.  i = i + 1
6.  End while
```

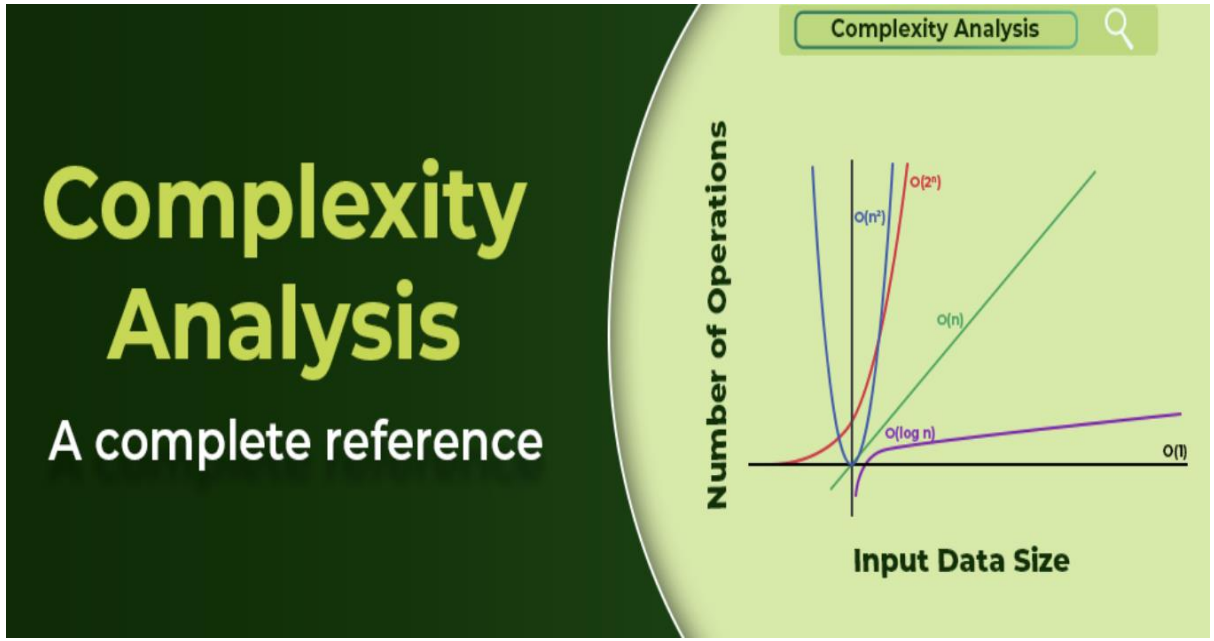
- To calculate time complexity, we must consider each line of code in the program.
- Consider the multiplication function as an example:
  - Let T(n) be a function of the algorithm's time complexity.
  - **Lines 1 and 2** have a time complexity of **O. (1).**
  - **Line 3 represents a loop**. As a result, you must repeat **lines 4 and 5 (n -1) times**. As a result, the time complexity of lines **4 and 5 is O. (n)**.
  - Finally, adding the time complexity of all the lines yields the overall time complexity of the multiple function fT(n) = O(n).

# Method to Calculate Space Complexity

```
1.  int mul, i
2.  While i < = n do
3.  mul <- mul * array[i]
4.  i <- i + 1
5.  end while
6.  return mul
```

- Let S(n) denote the algorithm's space complexity.
- In most systems, an integer occupies 4 bytes of memory. As a result, the number of allocated bytes would be the space complexity.
- Line 1 allocates memory space for two integers, resulting in S(n) = 4 bytes multiplied by 2 = 8 bytes. Line 2 represents a loop. Lines 3 and 4 assign a value to an already existing variable. As a result, there is no need to set aside any space. The return statement in line 6 will allocate one more memory case. As a result, S(n)= 4 times 2 + 4 = 12 bytes.
- Because the array is used in the algorithm to allocate n cases of integers, the final space complexity will be S(n) = n + 12 = O (n).

# Types of Time Complexity




- Constant Time
- Linear Time
- Logarithmic Time
- Quadratic Time

# Types of Time Complexity: Constant

- When an algorithm is not reliant on the input size n, it is said to have constant time with order **O (1).**
- The run time will always be the same, regardless of the input size n.

```
main()
{
int a;
printf("Enter value of a=");
scanf("%d",&a);
printf("a= %d", a);
}
```

# Types of Time Complexity: Linear

- When the running time of an algorithm rises linearly with the length of the input, it is said to have linear time complexity.
- When a function checks all the values in an input data set, it is said to have Time complexity of order **O(n)**.

```
main()
{
int a[5]={2, 3, 5, 7, 9};
printf("Elements of a=\n");
for(int i=0;i<5;i++)
    printf("a= %d", a);
}
```

# Types of Time Complexity: Quadratic

- When the execution time of an algorithm **rises** non-linearly ($n^2$) with the length of the input, it is said to have a quadratic time complexity.
- In general, nested loops fall into the quadratic time complexity order, where one loop takes O (n) and if the function contains loops inside loops, it takes O(n)*O(n)=O($n^2$)

```
for(int i=0; i<n; i++)
{
for(int j=0; j<n; j++)
 {
    printf("%d",arr[i][j]);
 }
}
```

# Types of Time Complexity: Logarithmic

- When an algorithm **lowers** the amount of the input data in each step, it is said to have a logarithmic time complexity **O (log n).**
- Binary search and Binary tree functions are some of the algorithms with logarithmic time complexity.

```
int binarySearch(int arr[], int l, int r, int x)
{
   if (r >= l) {
      int mid = l + (r - l) / 2;
       if (arr[mid] == x)
         return mid;
       if (arr[mid] > x)
         return binarySearch(arr, l, mid - 1, x);
         return binarySearch(arr, mid + 1, r, x);
   }}
```

# Evaluation of Algorithms
# Asymptotic Notations in Complexity Analysis

- In mathematics, **asymptotic analysis,** also known as asymptotics, is a method of describing the limiting behaviour of a function.
- **In computing**, asymptotic analysis of an algorithm refers to defining the **mathematical boundation of its run-time performance based on the input size.**
- **For example,** the running time of one operation is computed as **f(n),** and maybe for another operation, it is computed as **g($2^n$).**
- This means the first operation's running time will increase **linearly** with the increase in, "n" and the running time of the second operation will increase **exponentially when n increases.** Similarly, the running time of both operations will be nearly the same if n is small in value.

# Types of Asymptotic Notations

- Best Case (Omega Notation (Ω))
- Average Case (Theta Notation (Θ))
- Worst Case (Big-O Notation(O))

# Best Case (Omega Notation (Ω))

- **Omega (Ω)** notation specifies the **asymptotic lower bound for a function f(n).**
- Follow the steps below to calculate **Ω** for a program:
  - Break the program into smaller segments.
  - **Find the number of operations performed** for each segment(in terms of the input size), assuming the given input is such that the program takes the **least amount of time.**
  - Add up all the operations and simplify it
  - **Remove all the constants and choose the term having the least order** or any other function which is always less than f(n) when n tends to infinity, let's say it is as Ω(f(n)).
  - Omega notation doesn't really help to analyze an algorithm because it is not useful to evaluate an algorithm for the best cases of inputs.
  - **For example: How we select lower bound of a function that takes least amount of time**
    - **T(n) = $73n^3 + 22n^2 + 58$ -> T(n) = Ω $(n^2)$**

# Average Case (Theta Notation (Θ))

- When considering a function **T(n) = $73n^3 + 22n^2 + 58$**, tight bound (Upper and lower), average-case (exact growth rate)
- If an algorithm runs in **Θ($n^2$)** time, then it means **it will always take** some constant times **$n^2$** operations in both best and worst cases.
- Avoiding all constants and will be written in Big Theta notation as **T(n) = Θ($n^3$)**, that means ***T*(*n*) grows asymptotically as fast as $n^3$**
- Follow the steps below to calculate Θ for a program:
  - Break the program into smaller segments.
  - Find all types of inputs and calculate the number of operations they take to be executed. Make sure that the input cases are equally distributed.
  - Find the sum of all the calculated values and divide the sum by the total number of inputs let say the function of n obtained is g(n) after removing all the constants, then in Θ notation, it's represented as Θ(g(n)).

# Worst Case (Big-O Notation(O))

- **Big – O (O) notation** specifies the **asymptotic upper bound** for a function f(n).
- Follow the steps below to calculate O for a program:
  - Break the program into smaller segments.
  - Find the number of operations performed for each segment (in terms of the input size) assuming the given input is such that the program takes the **maximum time i.e the worst-case scenario.**
  - Add up all the operations and simplify it, let's say it is f(n).
  - **Remove all the constants and choose the term having the highest order** because for n tends to infinity the constants and the lower order terms in f(n) will be insignificant, let say the function is g(n) then, big-O notation is O(g(n)).
  - It is the **most widely used notation** as it is easier to calculate since there is no need to check for every type of input as it was in the case of theta notation, also since the worst case of input is taken into account it pretty much gives the upper bound of the time the program will take to execute.

```
int fib(int n)
{
    int f[n + 2];          +1
    int i;                 +1
    f[0] = 0;              +1
    f[1] = 1;              +1

    for(i = 2; i <= n; i++)   +n
    {
      f[i] = f[i - 1] + f[i - 2];
    }

    return f[n];           +1

}
```

Total Time Taken = n + 5

Time Complexity = Ω(n+5) = Ω(n)

# Real-Life Analogy for Asymptotic Notations

Imagine you're driving on a **highway**  with a **speed limit**:

- **Θ(60 km/h) speed limit means:**
  - You must drive **at least 50 km/h** (lower bound).
  - You can't go faster than **70 km/h** (upper bound).
  - Your speed is **always within this range** (tight bound).

- **Big-O (O) analogy:** "You can drive **at most** 70 km/h." (Upper bound)
- **Big-Omega (Ω) analogy:** "You must drive **at least** 50 km/h." (Lower bound)
- **Theta (Θ) analogy:** "Your speed is **always between 50 and 70 km/h.**" (Tight bound)